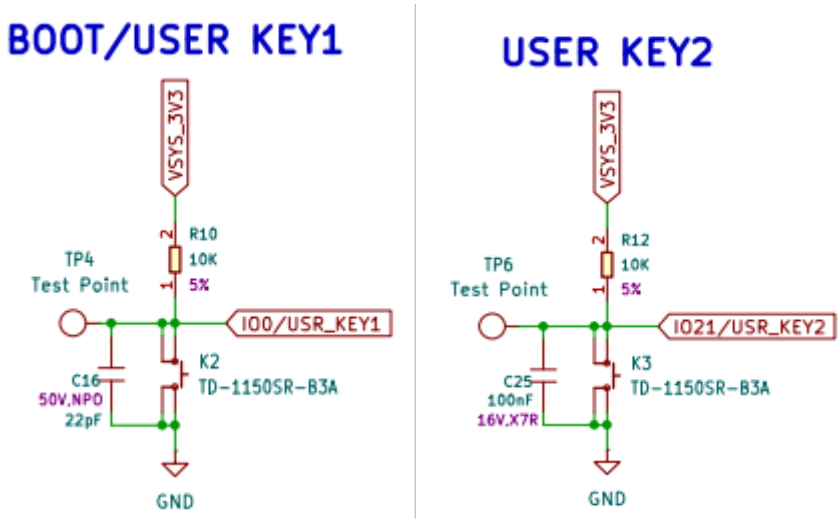
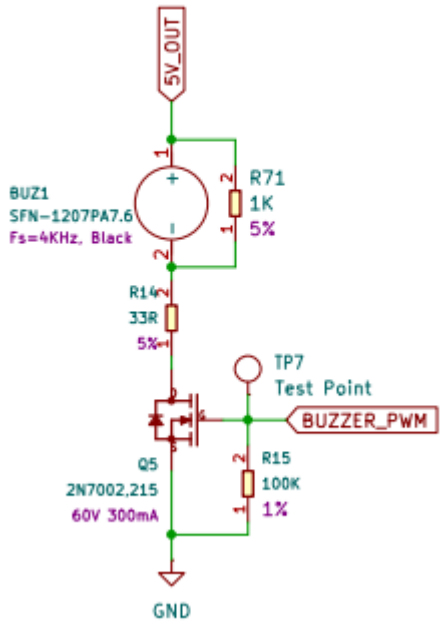
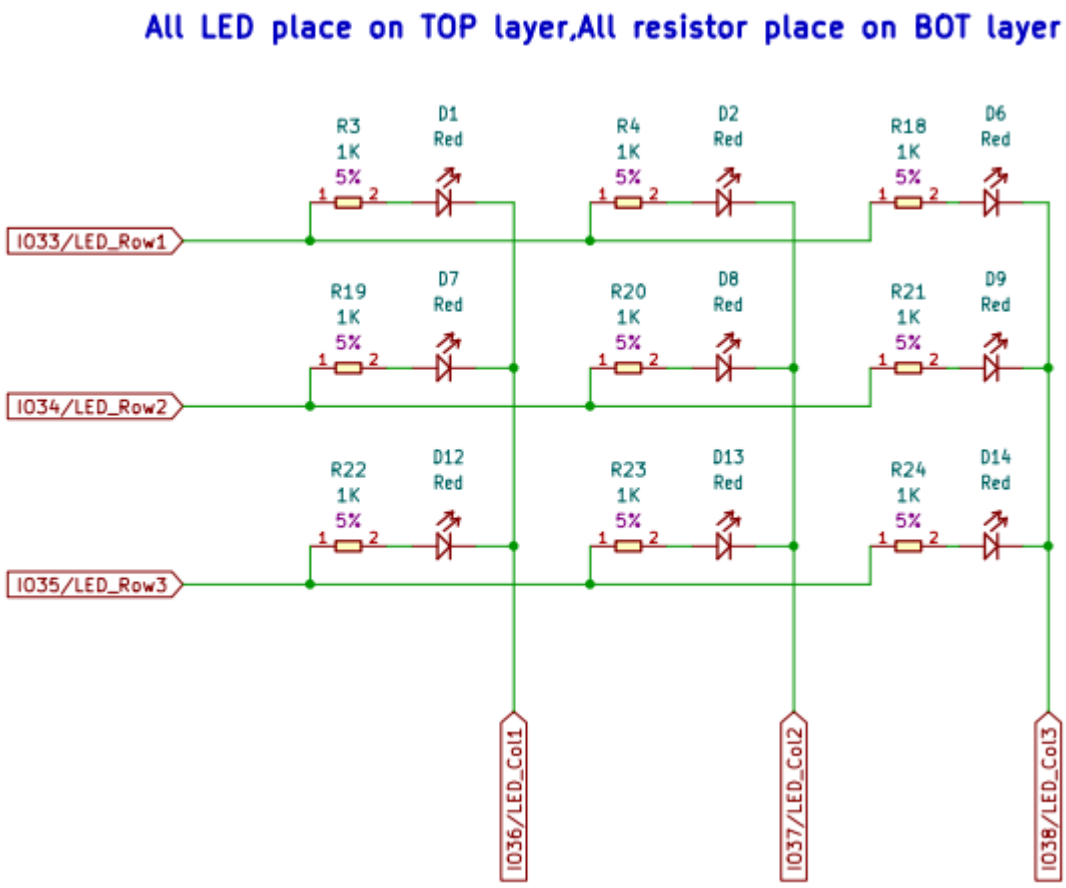
请输入标题

一、 编程指南

按键，蜂鸣器与LED







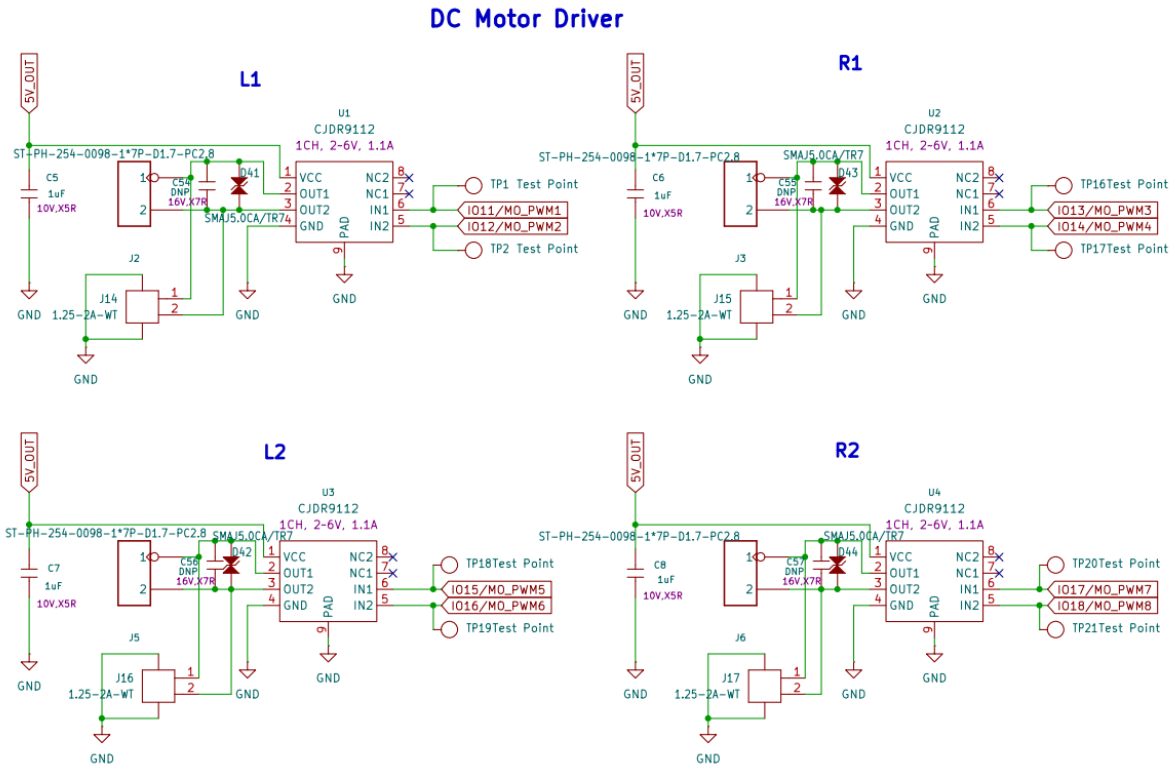
由原理图可知，按键A的引脚为21，按键B的引脚为0，蜂鸣器引脚为26，LED矩阵为3行3列，行：引脚33，34，35，列：引脚36，37，38。当行引脚为高电平，列引脚为低电平时，LED亮起。

将以下代码复制进Arduino IDE 草图，用数据线将主板与电脑连接，保存草图后选择XIAO_ESP32S3开发板及端口，点击上传：

```
#define USER_KEY1 0
#define USER_KEY2 21
#define LED_ROW1 33
#define LED_COL1 36
#define USER_BUZZER 26
void setup(){
  // 初始化按键，LED灯
  pinMode(USER_KEY1,INPUT);
  pinMode(USER_KEY2,INPUT);
  pinMode(USER_BUZZER, OUTPUT);
  pinMode(LED_ROW1, OUTPUT);
  pinMode(LED_COL1, OUTPUT);
}
void loop() {
  if (digitalRead(USER_KEY1) == LOW){
    digitalWrite(LED_ROW1, HIGH);
      analogWrite(USER_BUZZER, 128);
  }
  if (digitalRead(USER_KEY2) == LOW){
    digitalWrite(LED_ROW1, LOW);
      analogWrite(USER_BUZZER, 0);
  }
}
```

成功上传程序后，按下按键B左下角LED灯亮起，蜂鸣器响起；按下按键A则熄灭，蜂鸣器熄声。

电机



由原理图可知：

马达1接口：

电机1：引脚11，负责左前车轮前进

电机2：引脚12，负责左前车轮后退

马达2接口：

电机3：引脚13，负责右前车轮后退

电机4：引脚14，负责右前车轮前进

马达3接口：

电机5：引脚15，负责左后车轮前进

电机6：引脚16：负责左后车轮后退

马达4接口：

电机7：引脚17，负责右前车轮后退

电机8：引脚18：负责右前车轮前进

将以下代码复制进Arduino IDE 草图，将马达连接至主板，用数据线将主板与电脑连接，保存草图后选择XIAO_ESP32S3开发板及端口，点击上传：

```
#include <Arduino.h>
#define MOTOR_PWM1 11
#define MOTOR_PWM2 12
```

```
#define MOTOR_PWM3 13
#define MOTOR_PWM4 14
#define MOTOR_PWM5 15
#define MOTOR_PWM6 16
#define MOTOR_PWM7 17
#define MOTOR_PWM8 18
void setup() {
  // 初始化串口
  Serial.begin(9600);
  // 初始化点击引脚
  pinMode(MOTOR_PWM1, OUTPUT);
  pinMode(MOTOR_PWM2, OUTPUT);
  pinMode(MOTOR_PWM3, OUTPUT);
  pinMode(MOTOR_PWM4, OUTPUT);
  pinMode(MOTOR_PWM5, OUTPUT);
  pinMode(MOTOR_PWM6, OUTPUT);
  pinMode(MOTOR_PWM7, OUTPUT);
  pinMode(MOTOR_PWM8, OUTPUT);
  analogWriteFrequency(5000);
}
void loop() {
    analogWrite(MOTOR_PWM1, 255); //数值为0~255，对应车轮最小速度~最大速度
    analogWrite(MOTOR_PWM2, 0);
    analogWrite(MOTOR_PWM3, 0);
    analogWrite(MOTOR_PWM4, 255);
    analogWrite(MOTOR_PWM5, 255);
    analogWrite(MOTOR_PWM6, 0);
    analogWrite(MOTOR_PWM7, 0);
    analogWrite(MOTOR_PWM8, 255);
}
```

成功上传程序后，可以观察到小车以直线行驶。

舵机

机械臂使用说明

主板机械臂插座的舵机1引脚为47，舵机2引脚为48，因为步进电机已将ESP32S3的pwm资源全部使用完毕，所以舵机pwm将使用软件模拟。

将以下代码复制进Arduino IDE 草图，将机械臂连接至主板，用数据线将主板与电脑连接，保存草图后选择XIAO_ESP32S3开发板及端口，点击上传：

```
#include <Arduino.h>
#define STEER_PIN1 47
#define STEER_PIN2 48
hw_timer_t * time_servo1 = NULL;
volatile uint16_t pwmValue1 = 128;
volatile uint16_t pwmValue2 = 128;
#define PWM_PERIOD 255
void setup() {
  pinMode(STEER_PIN1, OUTPUT);
  pinMode(STEER_PIN2, OUTPUT);
  time_servo1 = timerBegin(1, 80, true);
  // 配置时钟中断函数
  timerAttachInterrupt(time_servo1, &func_servo, true);
  // 设置时钟周期为1ms
  timerAlarmWrite(time_servo1, 100, true);
  // 使能时钟中断
  timerAlarmEnable(time_servo1);
```

```
}
void loop() {
  pwmValue1 = 13;
  for (pwmValue2 = 4; pwmValue2 <= 10; pwmValue2++){    // 机械臂舵机控制，前决定初始位置，后决定落下位置（越小越高）
    delay(50);
  }
  delay(1000);
  for (pwmValue1 = 13; pwmValue1 >=6; pwmValue1--){     // 夹子舵机控制，前决定初始位置，后决定合并位置（越小越紧）
    delay(50);
  }
  delay(1000);
  for (pwmValue2 = 10; pwmValue2 >= 4; pwmValue2--){
    delay(50);
  }
  delay(3000);
}
void func_servo(){
 static uint8_t pwmCount = 0;
  static uint8_t pwmCount2 = 0;
 if (++pwmCount >= pwmValue1) {
  digitalWrite(STEER_PIN1, LOW);  // 当计数达到设定的占空比时，输出低电平
 }
 if (pwmCount >= PWM_PERIOD) {
  pwmCount = 0;
  digitalWrite(STEER_PIN1, HIGH);  // 当计数达到PWM周期时，重置计数并输出高电平
 }
 if (++pwmCount2 >= pwmValue2) {
  digitalWrite(STEER_PIN2, LOW);
 }
 if (pwmCount2 >= PWM_PERIOD) {
  pwmCount2 = 0;
  digitalWrite(STEER_PIN2, HIGH);
 }
}
```

程序成功上传后，将看到机械臂进行反复夹取动作。

光线传感器

光线传感器输出数据通过模拟接口传输至主板，grove接口可实现即插即用，跳过硬件焊接或杜邦线连接步骤。

将以下代码复制进Arduino IDE 草图，将光线传感器连接至主板的模拟接口，用数据线将主板与电脑连接，保存草图后选择XIAO_ESP32S3开发板及端口，点击上传：

```
#define LIGHT 7 // 定义光线传感器连接的引脚为7
void setup() {
 Serial.begin(9600);  // 设置串口通信速率为9600波特，用于后续将传感器数据打印到串行监视器上
 pinMode(LIGHT, INPUT);  // 将引脚7设置为输入模式，以读取光线传感器的模拟输入值
}
void loop() {
 int sensorValue = analogRead(LIGHT);  // 函数从引脚7读取模拟输入值，并将其存储在变量 sensorValue 中
 Serial.print("Light value:");
 Serial.println(sensorValue);  //  将读取到的传感器值打印到串行监视器上
 delay(100);  // 每次读取间隔100ms
}
```

上传成功后，打开串口监视器，串口自动打印管线传感器的数据

输出　串口监视器 ×

消息（按回车将消息发送到"COM7"上的"XIAO_ESP32S3"）

```
19:37:19.084 -> Light value:116
19:37:19.179 -> Light value:117
19:37:19.272 -> Light value:116
19:37:19.365 -> Light value:117
19:37:19.445 -> Light value:116
19:37:19.579 -> Light value:121
19:37:19.657 -> Light value:133
19:37:19.744 -> Light value:160
19:37:19.875 -> Light value:179
19:37:19.968 -> Light value:183
19:37:20.046 -> Light value:185
19:37:20.186 -> Light value:187
19:37:20.278 -> Light value:189
19:37:20.345 -> Light value:191
19:37:20.480 -> Light value:192
19:37:20.572 -> Light value:197
19:37:20.665 -> Light value:198
19:37:20.744 -> Light value:199
```

声音传感器

声音传感器输出数据通过模拟接口传输至主板，grove接口可实现即插即用，跳过硬件焊接或杜邦线连接步骤。

将以下代码复制进Arduino IDE 草图，将声音传感器连接至主板的模拟接口，用数据线将主板与电脑连接，保存草图后选择XIAO_ESP32S3开发板及端口，点击上传。

```
#define SOUND 7
void setup() {
  // 初始化串口和引脚
  Serial.begin(9600);
   pinMode(SOUND, INPUT);
}
void loop() {
  // 读取引脚7的输入
  int sensorValue = analogRead(SOUND);
  // 串口打印读取到的数据
  Serial.print("Sound value:");
  Serial.println(sensorValue);
  delay(100);  // 每次读取间隔100ms
}
```

上传成功后，打开串口监视器，串口自动打印声音传感器的数据：

输出　串口监视器 ×

消息（按回车将消息发送到"COM7"上的"XIAO_ESP32S3"）

```
20:12:13.380 -> Sound value:27
20:12:13.467 -> Sound value:0
20:12:13.599 -> Sound value:0
20:12:13.691 -> Sound value:855
20:12:13.784 -> Sound value:229
20:12:13.867 -> Sound value:0
20:12:14.003 -> Sound value:218
20:12:14.067 -> Sound value:125
20:12:14.191 -> Sound value:0
20:12:14.315 -> Sound value:493
20:12:14.409 -> Sound value:0
20:12:14.501 -> Sound value:541
20:12:14.596 -> Sound value:0
20:12:14.667 -> Sound value:4
20:12:14.798 -> Sound value:550
20:12:14.890 -> Sound value:0
20:12:14.967 -> Sound value:367
20:12:15.107 -> Sound value:613
20:12:15.200 -> Sound value:485
```

加速度传感器

加速度传感器输出数据通过I2C接口传输至主板，grove接口可实现即插即用，跳过硬件焊接或杜邦线连接步骤。

下载 Grove-3-Axis-Digital-Accelerometer-2g-to-16g-LIS3DHTR.zip 到arduino库

将以下代码复制进Arduino IDE 草图，将加速度传感器连接至主板的I2C接口，用数据线将主板与电脑连接，保存草图后选择XIAO_ESP32S3开发板及端口，点击上传：

```
#include "LIS3DHTR.h"
#include <Wire.h>
LIS3DHTR<TwoWire> LIS; //IIC
#define I2C_SDA 39
#define I2C_SCL 40
#define WIRE Wire
void setup() {
  Serial.begin(9600);
  Wire.begin(I2C_SDA, I2C_SCL);
  // 初始化加速度传感器
  LIS.begin(Wire,0x19); //IIC init
  LIS.openTemp();  //If ADC3 is used, the temperature detection needs to be turned off.
  delay(100);
  LIS.setFullScaleRange(LIS3DHTR_RANGE_2G);
  LIS.setOutputDataRate(LIS3DHTR_DATARATE_50HZ);
}
void loop() {
    if (!LIS) {
        Serial.println("LIS3DHTR didn't connect.");
        while (1);
        return;
    }
    //3 axis
    Serial.print("x:"); Serial.print(LIS.getAccelerationX()); Serial.print("  ");
    Serial.print("y:"); Serial.print(LIS.getAccelerationY()); Serial.print("  ");
    Serial.print("z:"); Serial.println(LIS.getAccelerationZ());
    delay(500);
```

}

上传成功后，打开串口监视器，串口自动打印加速度传感器的数据：

```
输出    串口监视器 ×
消息 (按回车将消息发送到"COM7"上的"XIAO_ESP32S3")
20:27:19.261 -> x:-0.22  y:-0.45  z:0.85
20:27:19.735 -> x:0.19  y:-0.24  z:0.47
20:27:20.242 -> x:0.52  y:-0.28  z:0.66
20:27:20.769 -> x:0.50  y:-0.40  z:0.79
20:27:21.267 -> x:0.50  y:-0.58  z:0.80
20:27:21.751 -> x:0.47  y:-0.50  z:0.88
20:27:22.280 -> x:0.44  y:-0.44  z:0.80
20:27:22.777 -> x:0.50  y:-0.44  z:0.77
20:27:23.306 -> x:0.50  y:-0.42  z:0.78
20:27:23.766 -> x:0.49  y:-0.46  z:0.73
20:27:24.270 -> x:0.48  y:-0.46  z:0.79
20:27:24.814 -> x:0.49  y:-0.44  z:0.79
20:27:25.278 -> x:0.49  y:-0.44  z:0.74
20:27:25.829 -> x:0.49  y:-0.44  z:0.78
20:27:26.287 -> x:0.49  y:-0.45  z:0.79
20:27:26.810 -> x:0.49  y:-0.43  z:0.82
20:27:27.294 -> x:0.46  y:-0.43  z:0.77
20:27:27.799 -> x:0.49  y:-0.44  z:0.78
20:27:28.315 -> x:0.49  y:-0.44  z:0.80
20:27:28.842 -> x:0.51  y:-0.43  z:0.78
```

温湿度传感器

温湿度传感器输出数据通过数字传输至主板，grove接口可实现即插即用，跳过硬件焊接或杜邦线连接步骤。

下载 Grove_Temperature_And_Humidity_Sensor.zip 到arduino库

导航至“文件”>“示例”>“Grov Temperature And Humidity Sensor ”>“DHTtester”，打开该例子，做如下修改：

```
#define DHTTYPE DHT11   // DHT 11
//#define DHTTYPE DHT22   // DHT 22  (AM2302)
#define DHTPIN 3
//#define DHTPIN 2
```

将温湿度传感器连接至主板的数字接口，用数据线将主板与电脑连接，选择XIAO_ESP32S3开发板及端口，点击上传。

上传成功后，打开串口监视器，串口自动打印温湿度传感器的数据：

```
输出    串口监视器 ×
消息 (按回车将消息发送到"COM7"上的"XIAO_ESP32S3")
20:25:06.151 -> Humidity: 62.00 %       Temperature: 25.50 *C
20:25:07.771 -> Humidity: 52.00 %       Temperature: 29.40 *C
20:25:09.782 -> Humidity: 51.00 %       Temperature: 29.50 *C
20:25:11.808 -> Humidity: 52.00 %       Temperature: 29.50 *C
20:25:13.813 -> Humidity: 52.00 %       Temperature: 29.50 *C
20:25:15.835 -> Humidity: 51.00 %       Temperature: 29.50 *C
20:25:17.896 -> Humidity: 51.00 %       Temperature: 29.50 *C
20:25:19.885 -> Humidity: 51.00 %       Temperature: 29.60 *C
20:25:21.941 -> Humidity: 51.00 %       Temperature: 29.70 *C
20:25:23.934 -> Humidity: 50.00 %       Temperature: 29.70 *C
20:25:25.999 -> Humidity: 50.00 %       Temperature: 29.70 *C
20:25:27.984 -> Humidity: 50.00 %       Temperature: 29.70 *C
20:25:30.052 -> Humidity: 50.00 %       Temperature: 29.70 *C
20:25:32.052 -> Humidity: 50.00 %       Temperature: 29.70 *C
20:25:34.079 -> Humidity: 50.00 %       Temperature: 29.70 *C
20:25:36.086 -> Humidity: 49.00 %       Temperature: 29.70 *C
```

超声波传感器

超声波传感器输出数据通过数字接口传输至主板，grove接口可实现即插即用，跳过硬件焊接或杜邦线连接步骤。

下载 Grove_Ultrasonic_Ranger.zip 到arduino库

导航至“文件”>“示例”>“Grove Ultrasonic Ranger”>“UltrasonicDisplayOnTerm”，打开该例子，将超声波传感器连接至主板的数字接口，用数据线将主板与电脑连接，选择XIAO_ESP32S3开发板及端口，点击上传。

上传成功后，打开串口监视器，串口自动打印该时刻超声波传感器检测到的障碍物距离。

```
输出    串口监视器 ×
消息 (按回车将消息发送到"COM7"上的"XIAO_ESP32S3")
20:33:11.212 -> The distance to obstacles in front is:
20:33:11.253 -> 4 inch
20:33:11.464 -> 10 cm
20:33:11.737 -> The distance to obstacles in front is:
20:33:11.737 -> 4 inch
20:33:11.965 -> 3 cm
20:33:12.224 -> The distance to obstacles in front is:
20:33:12.224 -> 1 inch
20:33:12.474 -> 10 cm
20:33:12.755 -> The distance to obstacles in front is:
20:33:12.755 -> 4 inch
20:33:13.003 -> 10 cm
20:33:13.236 -> The distance to obstacles in front is:
20:33:13.236 -> 4 inch
20:33:13.516 -> 10 cm
20:33:13.738 -> The distance to obstacles in front is:
20:33:13.738 -> 4 inch
```

注意：超声波最好不要连接D5接口使用，以免造成数据冲突

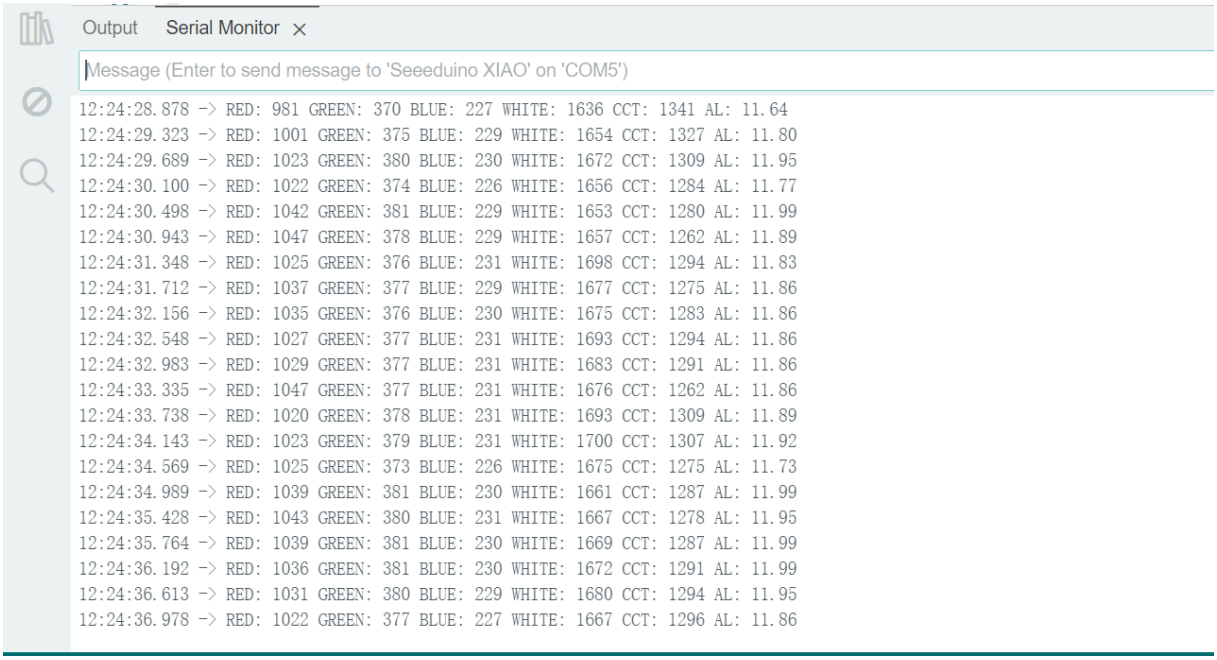
颜色识别传感器

颜色识别传感器输出数据通过I2C接口传输至主板，grove接口可实现即插即用，跳过硬件焊接或杜邦线连接步骤。

下载 VEML6040.zip 到arduino库

导航至“文件”>“示例”>“VEML6040”>“auto_mode”，打开该例子，将颜色识别传感器连接至主板的I2C接口，用数据线将主板与电脑连接，选择XIAO_ESP32S3开发板及端口，点击上传。

上传成功后，打开串口监视器，串口自动打印该时刻传感器检测到的颜色的RGBW值。



循迹传感器

循迹传感器输出数据通过数字接口传输至主板，grove接口可实现即插即用，跳过硬件焊接或杜邦线连接步骤。

将以下代码复制进Arduino IDE 草图，将循迹传感器连接至主板的数字接口，用数据线将主板与电脑连接，保存草图后选择XIAO_ESP32S3开发板及端口，点击上传：

```
#define LEFT 1
#define RIGHT 2
int towar_left = 0;
int towar_right = 0;
void setup() {
  // 初始化串口和引脚
  Serial.begin(9600);
  pinMode(LEFT, INPUT);
  pinMode(RIGHT, INPUT);
}
void loop() {
  towar_left=digitalRead(LEFT);
  towar_right=digitalRead(RIGHT);
  Serial.print("line follower : left ");
  Serial.print(towar_left);
  Serial.print("  right ");
```

```
  Serial.println(towar_right);
}
```

上传成功后，打开串口监视器，串口自动打印此时两个红外传感器的值。

```
输出    串口监视器 ×
消息 (按回车将消息发送到"COM7"上的"XIAO_ESP32S3")

20:47:03.593 -> line follower : left 1  right 1
20:47:03.593 -> line follower : left 1  right 1
20:47:03.593 -> line follower : left 1  right 1
20:47:03.593 -> line follower : left 1  right 1
20:47:03.593 -> line follower : left 1  right 1
20:47:03.593 -> line follower : left 1  right 1
20:47:03.593 -> line follower : left 1  right 1
20:47:03.593 -> line follower : left 1  right 1
20:47:03.593 -> line follower : left 1  right 1
20:47:03.593 -> line follower : left 1  right 1
20:47:03.593 -> line follower : left 1  right 1
20:47:03.593 -> line follower : left 1  right 1
20:47:03.593 -> line follower : left 1  right 1
20:47:03.593 -> line follower : left 1  right 1
20:47:03.593 -> line follower : left 1  right 1
```

二、示例

巡线与障碍物检测夹取

硬件连接

将循迹传感器连接至主板的数字接口GROVE 4,

将颜色识别传感器连接至主板的I2C接口GROVE 5,

将超声波传感器连接至主板的数字接口GROVE1，

将马达连接至主板。

上传程序

将以下代码复制进Arduino IDE 草图，用数据线将主板与电脑连接，保存草图后选择XIAO_ESP32S3开发板及端口，点击上传：

followline_colorrecognition_ultrasonic_final(1) (1).zip

注意

XIAO ESP32S3 的板载软件包至少需要 2.0.8 版本。

如果程序编译成功上传失败，让开发板进入BOOT模式再重新上传：按住按键B，按压复位键，再松开按键B

程序分析

这个程序控制智能小车通过多种传感器和舵机实现巡线、避障、颜色识别等功能。通过FreeRTOS 的任务机制，实现了多任务并行处理。

全局变量和引脚定义

- 舵机引脚定义：`STEER_PIN1` 和 `STEER_PIN2` 控制两个舵机，舵机用于机械臂的控制。
- 巡线模块引脚定义：`LEFT` 和 `RIGHT` 分别用于读取左、右巡线传感器的值。
- 用户按键和蜂鸣器：`USER_KEY1` 定义为按键引脚，`USER_BUZZER` 定义为蜂鸣器引脚。
- 电机引脚定义：`MOTOR_PWM1` 到 `MOTOR_PWM8` 分别用于控制小车的八个电机。
- 超声波雷达：`Ultrasonic ultrasonic(5)` 初始化超声波传感器，测量距离。
- 颜色识别传感器：`VEML6040 RGBWSensor` 初始化颜色传感器。
- I2C 引脚定义：`I2C_SDA` 和 `I2C_SCL` 分别为 I2C 数据线和时钟线。

RTOS任务

- 巡线任务**ColorTask**：该任务每隔50ms读取一次颜色传感器和超声波传感器的数据，并控制LED矩阵的显示状态。任务在核心0上运行。

setup 函数

- 初始化串口通信、I2C 通信、颜色传感器，并设置传感器的配置。
- 设置巡线传感器、LED 矩阵、蜂鸣器、舵机、电机引脚为输入/输出模式。
- 初始化蜂鸣器和舵机的定时器，配置定时器中断函数和时钟周期。
- 创建并运行巡线任务ColorTask。

loop 函数

- 主循环中，依次执行以下操作：
- func_motor()：根据巡线传感器的状态，控制电机的运行方向。
- 超声波测距：测量前方障碍物的距离，若距离小于8厘米，进行以下操作：
  - 蜂鸣器鸣叫
  - 停止电机
  - 控制机械臂的舵机进行抓取动作

功能函数

- motor_func：根据巡线传感器的状态，决定电机的运行方向（前进、左转、右转、后退）。
- motor_stop：停止所有电机。
- motor_go：使所有电机前进。
- motor_back：使所有电机后退。
- motor_left：使所有电机左转。
- motor_right：使所有电机右转。
- func_led：根据颜色传感器的最大颜色值，控制 LED 矩阵的显示状态。
- func_buzzer：根据设定的占空比，控制蜂鸣器的鸣叫。
- func_servo：根据设定的占空比，控制舵机的运行。

实现效果

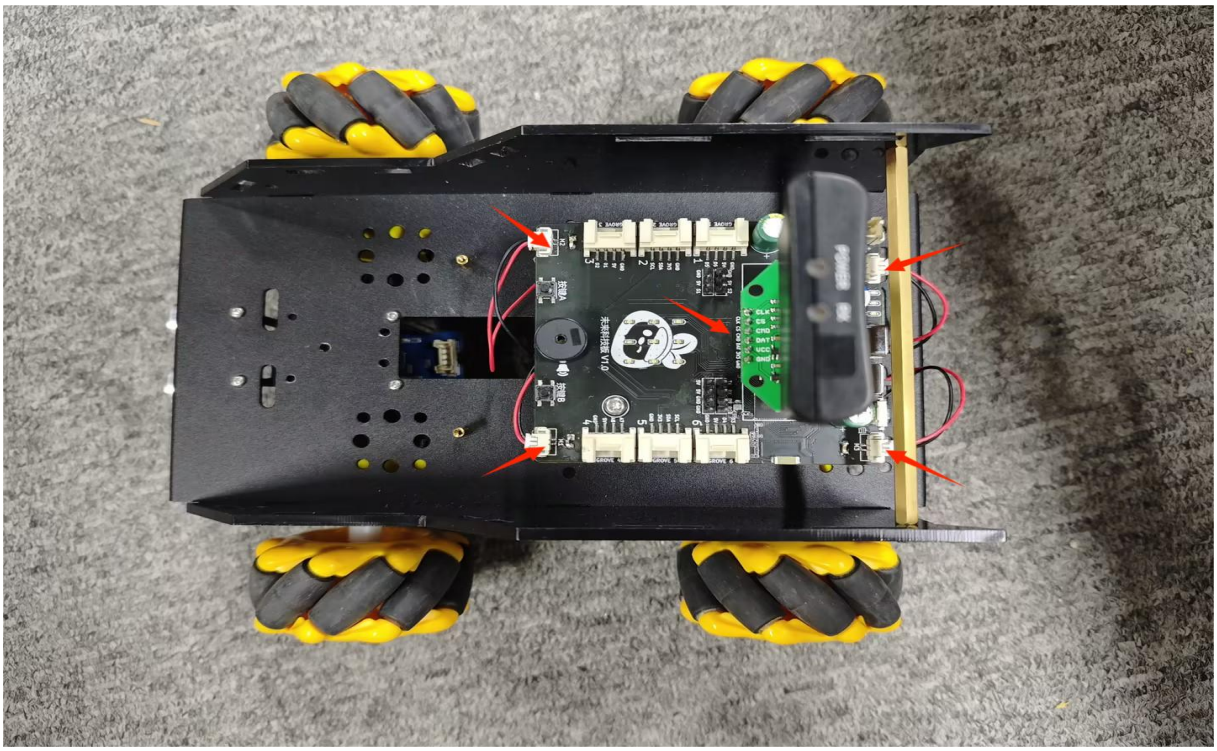
1. 巡线：读取左右巡线传感器的值，根据传感器状态控制电机的前进、左转、右转或后退。
2. 避障：通过超声波传感器测量前方障碍物距离，若距离小于8厘米，停止电机并进行机械臂的抓取动作。
3. 颜色识别：读取颜色传感器的 RGBW 值，根据颜色值控制 LED 矩阵的显示状态。
4. 蜂鸣器：根据设定的占空比，控制蜂鸣器的鸣叫。

PS2X

硬件连接

将PS2手柄接收器插到主板上，

将马达连接至主板上。



上传程序

下载 Arduino-PS2X-ESP32-master.zip 到arduino库，再将以下代码复制进Arduino IDE 草图，

PS2-V2(1).ino

用数据线将主板与电脑连接，保存草图后选择XIAO_ESP32S3开发板及端口，按住按键B，按压复位键，再松开按键B进入BOOT模式后，点击上传。

注意

XIAO ESP32S3 的板载软件包为 2.0.13 版本。

如果程序编译成功上传失败，让开发板进入BOOT模式再重新上传：按住按键B，按压复位键，再松开按键B

程序分析

全局变量和引脚定义

- 电机引脚定义：`MOTOR_PWM1` 到 `MOTOR_PWM8` 分别用于控制小车的八个电机。
- 定义了WiFi的SSID和密码，以及WiFi服务器实例 server。
- pwm_value[]: 预定义的PWM值，控制电机速度的不同档位。
- motor_pwm[]: 记录每个电机的当前PWM值索引。
- motor_pwm_num[][]: 记录电机对应的PWM引脚组合。
- PS2控制器对象：PS2_CLK，PS2_CS，PS2_CMD，PS2_DAT

setup 函数

- 初始化串口、设置电机PWM引脚和WiFi软AP。

- 配置PS2控制器，并设置定时器以每100ms读取一次PS2控制器的状态。

loop 函数

- 在每次循环中，将当前电机的PWM值应用到相应的引脚上，以控制电机的速度。

功能函数

- func_ps2：读取PS2控制器的数据，包括按键状态和模拟值。根据按钮的状态和压力调整电机PWM值，并打印相关信息。
- motor_change：根据传入的电机编号和方向，调整电机PWM值的增减，以实现电机速度的变化。

实现效果

可根据PS2控制器的按钮和摇杆输入被用来调整电机的速度和方向。

遥控器指令不匹配/遥控失效问题

如果您在使用中发现遥控器与接收器的指令不匹配或者无法正常控制小车时，通常是因为遥控器与无线接收器之间配对出错了。请尝试重新配对遥控器与接收器即可解决。

配对方法：

1. 先给主板烧一个带有遥控的程序 PS2-V2(1).ino

   左摇杆上下：前进/后退
   左摇杆左右：左甩尾/右甩尾
   右摇杆上下：向左/向右原地选择
   右摇杆左右：向左向右横移

   如果提示缺少library，请安装此library Arduino-PS2X-ESP32-master.zip ，无提示则略过
2. 将接收器插上主板，然后重启主板，此时接收器绿灯闪烁
3. 扣掉遥控器的电池，等待5秒后重新插入电池开机，此时遥控器红灯和绿灯会闪一下，等待1-3秒左右，接收器和遥控器红绿灯都常亮即配对完成。

10_1727247734.mp4

需要注意：

- 配对时周围只能开启一个接收器和一个遥控器的电源，否则配对过程中 遥控器与接收器会串台。
- 配对完成之后 请勿关闭遥控器的电源开关（接收器可正常断电关闭），遥控器一段时间不使用后会自动进入休眠模式，无需担心电池损耗。下次使用时只需要给接收器再次供电（小车开机），然后按下遥控器的“START”键，遥控器将自动连接到已配对的接收器上。
- 如果多台小车都需要进行遥控器配对，请在第一台配对完成之后，再开启第二台小车的配对过程，依次类推。配对过程中保持只有一个接收器和一个未配对的遥控器开启。